

PZ-VS1053 MP3 模块开发手册

第 1 章 音乐播放器实验

前几年，MP3 曾经风行一时，几乎人手一个。如果能自己做一个 MP3，我想对于很多朋友来说是一件十分骄傲的事情。我们 PZ-VS1053 模块内就集成了一颗非常强劲的 MP3 解码芯片：VS1053，利用该芯片我们可以实现 MP3/OGG/WMA/WAV/FLAC/AAC/MIDI 等各种音频文件的播放。本章我们将利用开发板与 PZ-VS1053 模块实现一个简单的 MP3 播放器。本章所要实现的功能为：开机先检测 SD 卡是否存在，如果检测无问题，则对 VS1053 进行 RAM 测试和正弦测试，测试完后开始循环播放 SD 卡 MUSIC 文件夹里面的歌曲（**必须在 SD 卡根目录建立一个 MUSIC 文件夹，并存放歌曲在里面**），并在 TFTLCD 上显示歌曲名字、播放时间、歌曲总时间、歌曲总数目、当前歌曲的编号等信息。KEY0 用于选择下一曲，KEY1 用于选择上一曲，KEY_UP 用来调节音量。DS0 用于指示程序运行状态本章分为如下几个部分：

- 1.1 PZ-VS1053 模块介绍
- 1.2 硬件设计
- 1.3 软件设计
- 1.4 实验现象

1.1 PZ-VS1053 模块介绍

1.1.1 特性参数

PZ-VS1053 MP3 模块是深圳普中科技有限公司推出的一款高性能音频编解码模块，该模块采用 VS1053B 作为主芯片，支持：

MP3/WMA/OGG/WAV/FLAC/MIDI/AAC 等音频格式的解码，并支持：OGG/WAV 音频格式的录音，支持高低音调节以及 EarSpeaker 空间效果设置，功能十分强大。

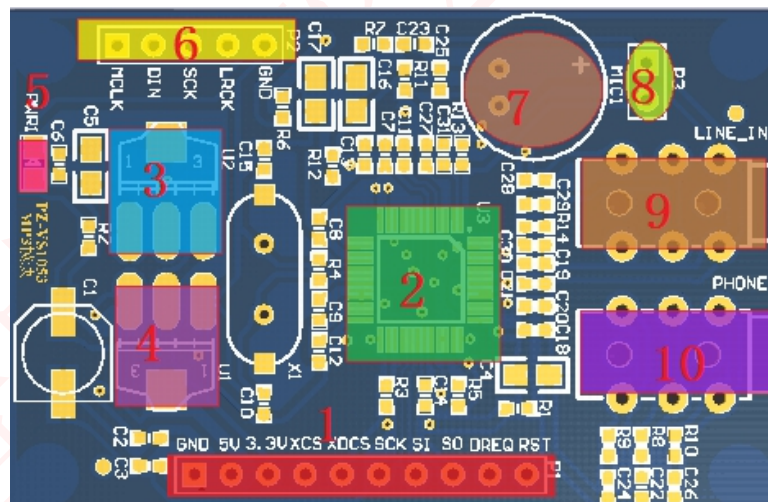
模块通过 SPI 接口与外部单片机通信，模块可以直接与 3.3V 单片机系统连接，也可以方便的与 5V 单片机系统连接（**特别注意：模块的信号线不能直接接 5V 单片机，如果要接，推荐在信号线上串联 1K 左右的电阻。**）。模块自带稳压芯片，外部仅需提供 5V/3.3V 电压即可，使用非常方便，该模块各参数如图所示：

项目	说明
接口特性	3.3V(串电阻后，可与 5V 系统连接)
解码格式	MP3、OGG、WMA、WAV、MIDI、AAC、FLAC（需要加载 patch）
编码格式	WAV(PCM/IMA ADPCM)、OGG（需要加载 patch）
对外接口	1 路 3.5mm 耳机接口、1 路 3.5mm LINE IN 接口、IIS 接口、供电及控制接口
板载录音	支持
工作温度	-30℃~85℃
其他特性	音量控制、高低音控制、EarSpeaker 空间效果、解码时间输出
模块尺寸	34.64mm*52.83mm

DAC 分辨率	18 位
总谐波失真 (THD)	0. 07%(Max)
动态范围 (A-加权)	100dB
信噪比	94dB
通道隔离度 (串扰)	80dB@600欧+GBUF 53dB@30欧+GBUF
咪头 (MIC) 放大增益	26dB
咪头 (MIC) 总谐波失真	0. 07%(Max)
咪头 (MIC) 信噪比	70dB
LINE IN 信号幅度	2800mVpp (Max)
LINE IN 总谐波失真	0. 014%(Max)
LINE IN 信噪比	90dB
LINE IN 阻抗	80K欧
工作电压	DC3. 3V/5. 0V (推荐 5. 0V 供电)
工作电流	15mA~40mA
Voh	2. 31V (Min)
Vol	0. 99V (Max)
Vih	1. 26V (Min)
Vil	0. 54V (Max)

1.1.2 模块介绍

PZ-VS1053 MP3 模块是深圳普中科技有限公司开发的一款高性能音频编解码模块，该模块接口丰富、功能完善，仅需提供电源（ 3. 3V/5. 0V），即可通过单片机（ 8/16/32 位单片机均可）控制模块实现音乐播放或者录音等功能， 模块资源图如图所示：



序号	模块资源
1	电源及SPI通信接口
2	VS1053B编解码芯片
3	1. 8V稳压芯片
4	3. 3V稳压芯片
5	模块电源指示灯
6	IIS输出接口
7	咪头 (MIC)
8	LINE IN/MIC选择接口
9	LINE IN接口
10	音频输出接口

从上图可以看出， PZ-VS1053 MP3 模块不但外观漂亮，而且功能齐全、接口丰富，模块尺寸为 34.64mm*52.83mm，并带有安装孔位，非常小巧，并且利于安装，可方便应用于各种设计。

PZ-VS1053 MP3 模块板载资源如下：

- ◆ 高性能编解码芯片： VS1053B
- ◆ 1 个 LINE IN/MIC 选择接口
- ◆ 1 个咪头
- ◆ 1 个电源指示灯（蓝色）
- ◆ 1 个 1.8V 稳压芯片
- ◆ 1 个 3.3V 稳压芯片
- ◆ 1 路 IIS 输出接口
- ◆ 1 路电源及 SPI 控制接口
- ◆ 1 路 3.5mm LINE IN 接口，支持双声道输入录音
- ◆ 1 路 3.5mm 音频输出接口，可直接插耳机

PZ-VS1053 模块采用高准设计，特点包括：

◆ 板载 VS1053B 高性能编解码芯片，支持众多音频格式解码，支持 OGG/WAV 编码。

- ◆ 板载稳压电路，仅需外部提供一路 3.3V 或 5V 供电即可正常工作；
- ◆ 板载 3.5mm 耳机插口，可直接插入耳机欣赏高品质音乐；
- ◆ 板载咪头（ MIC），无需外部麦克风，即可实现录音；
- ◆ 板载 IIS 输出，可以接外部 DAC，获得更高音质；
- ◆ 板载电源指示灯，上电状态一目了然；
- ◆ 采用国际 A 级 PCB 料，沉金工艺加工，稳定可靠；
- ◆ 采用全新元器件加工，纯铜镀金排针，坚固耐用；
- ◆ 人性化设计，各个接口都有丝印标注，使用起来一目了然；接口位置设计安排合理，方便顺手。

- ◆ PCB 尺寸为 34.64mm*52.83mm，并带有安装孔位，小巧精致。

1.1.3 模块引脚说明

PZ-VS1053 MP3 模块总共有 3 组排针：P1、P2 和 P3，均采用纯铜镀金排针，2.54mm 间距，方便与外部设备连接。

P1 排针为模块的供电与通信接口，采用 1*10P 排针，各引脚详细描述如图所示：

序号	名称	说明
1	GND	地
2	5V	5V 供电口，只可以供电
3	3.3V	3.3V 供电口，当使用 5V 供电的时候，这里可以输出 3.3V 电压给外部使用
4	XCS	片选输入（低有效）
5	XDCS	数据片选/字节同步
6	SCK	SPI 总线时钟线
7	SI	SPI 总线数据输入线
8	SO	SPI 总线数据输出线
9	DREQ	数据请求
10	RST	复位引脚（硬复位，低电平有效）

P2 排针为模块的 IIS 输出接口，采用 1*5P 排针，各引脚详细描述如图所示：

序号	名称	说明
1	MCLK	主时钟
2	DIN	数据输出
3	SCLK	位时钟
4	LRCK	帧时钟
5	GND	地

P3 排针为 LINE IN/MIC 选择接口，采用 1*2P 排针，各引脚详细描述如图所示：

序号	名称	说明
1	MIC	咪头正极信号
2	MICP/LINE1	咪头正极输入/线路输入 1

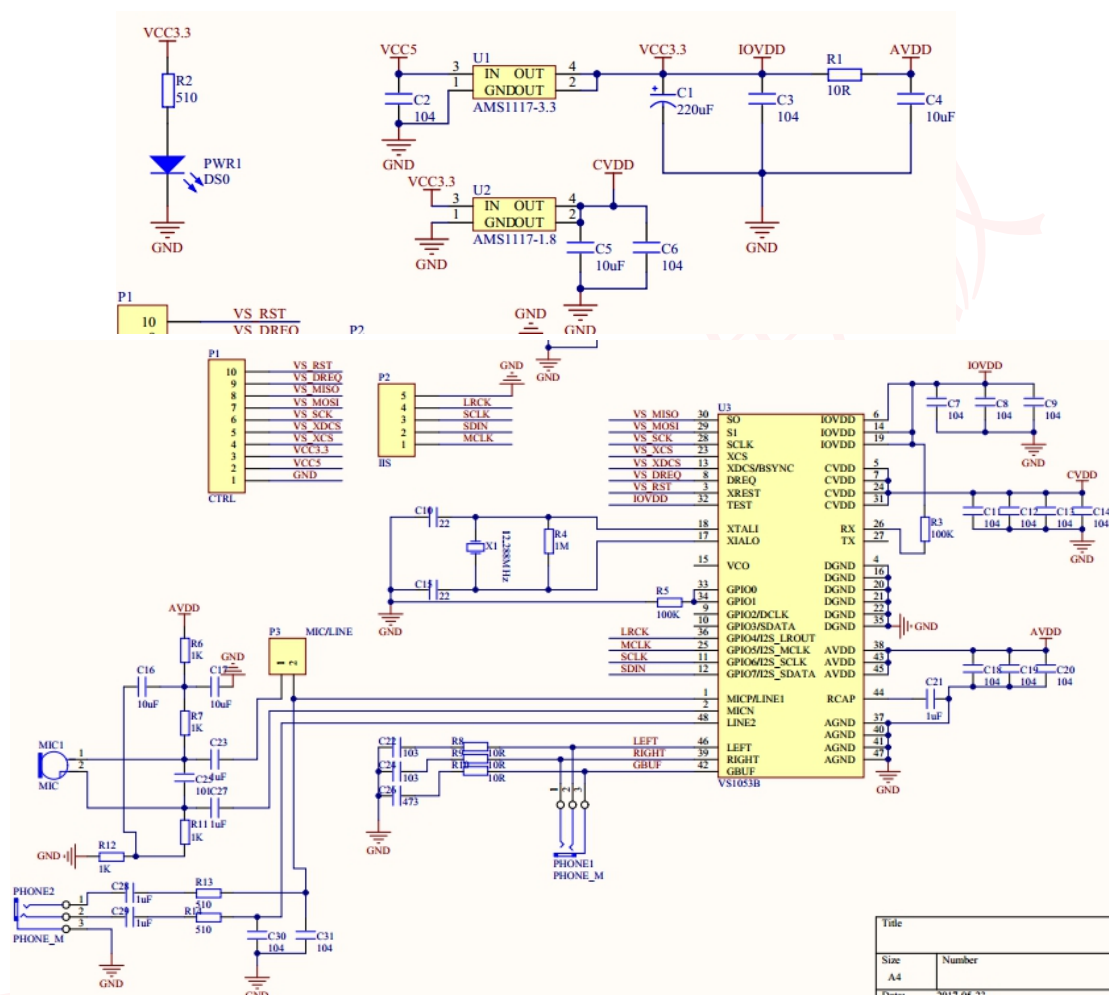
P3 接口不对外连接，当用跳线帽短接 P3 的 1 脚和 2 脚的时候（默认设置），咪头是直接连接在 VS1053 芯片上的，录音的时候，我们可以直接通过咪头实现声音采集。当不采用咪头拾音，而采用外部线路输入的时候，为了防止 MIC 拾音器对线路输入的影响，此时我们拔了 P3 的跳线帽即可。

1.1.4 模块使用

VS1053 通过 SPI 接口来接收输入的音频数据流，它可以是一个系统的从

机，也可以作为独立的主机。这里我们只把它当成从机使用。我们通过 SPI 口向 VS1053 不停的输入音频数据，它就会自动帮我们解码了，然后从输出通道输出音乐，这时我们接上耳机就能听到所播放的歌曲了。

PZ-VS1053 MP3 模块原理图如图所示：（看不清楚的话可以打开模块资料-原理图查看）



VS1053 通过 7 根线同 STM32 连接，他们是：VS_MISO、VS_MOSI、VS_SCK、VS_XCS、VS_XDCS、VS_DREQ 和 VS_RST。这 7 根线同 STM32 的连接关系如下：

VS_MISO(SO)：PA6
 VS_MOSI(SI)：PA7
 VS_SCK(SCK)：PA5
 VS_XCS(XCS)：PF7
 VS_XDCS(XDCS)：PF6
 VS_DREQ(DREQ)：PC13

VS_RST(RST): PE6

其中 VS_RST 是 VS1053 的复位信号线, 低电平有效。VS_DREQ 是一个数据请求信号, 用来通知主机, VS1053 可以接收数据与否。VS_MISO、VS_MOSI 和 VS_SCK 则是 VS1053 的 SPI 接口, 他们在 VS_XCS 和 VS_XDCS 下面来执行不同的操作。VS1053 的 SPI 是接在 STM32 的 SPI1 上面的。

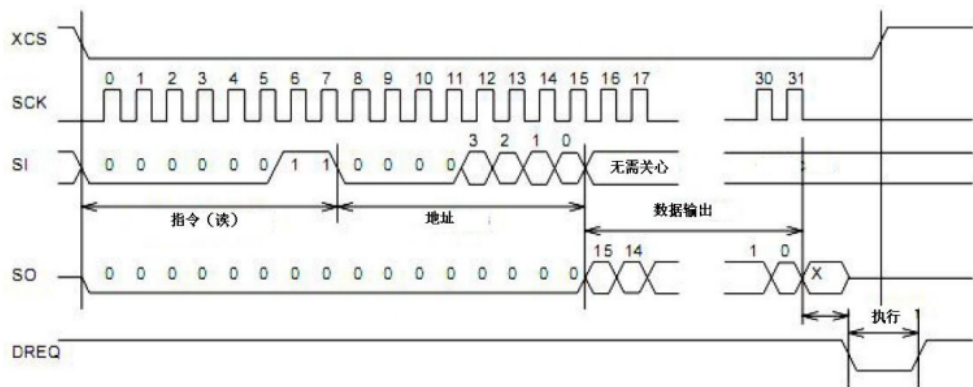
VS1053 的 SPI 支持两种模式: 1, VS1002 有效模式 (即新模式)。2, VS1001 兼容模式。这里我们仅介绍 VS1002 有效模式 (此模式也是 VS1053 的默认模式)。新模式下 VS1053 的 SPI 信号线功能描述如图所示:

SDI	SCI	描述
XDCS	XCS	SDI/SCI 片选信号, 低电平有效。高电平强制 SPI 进入 Standby 模式, 结束当前操作, 且 SO 变为高阻态。如果 SM_SDISHARE 位设置为 1, 则不使用 XDCS, 而有 XCS 内部反相后代替 XDCS。不过不推荐这种设置方式。
SCK		串行时钟输入。SCK 在传输的时候可以被打断, 但是必须保持 XCS/XDCS 低电平不变, 否则传输将中断。
SI		串行数据输入。在片选有效的情况下, SI 在 SCK 的上升沿处采样。所以, MCU 必须在 SCK 的下降沿上更新数据。
SO		串行数据输出。在读操作时, 数据在 SCK 的下降沿从此引脚输出, 在写操作时为高阻态。

VS1053 的 SPI 数据传送, 分为 SDI 和 SCI, 分别用来传输数据/命令。SDI 和前面介绍的 SPI 协议一样的, 不过 VS1053 的数据传输是通过 DREQ 控制的, 主机在判断 DREQ 有效 (高电平) 之后, 直接发送即可 (一次可以发送 32 个字节)。

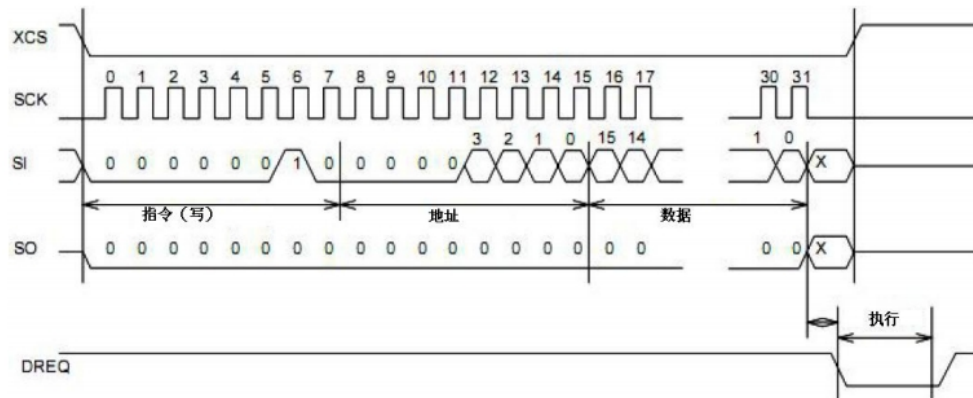
这里我们重点介绍一下 SCI。SCI 串行总线命令接口包含了一个指令字节、一个地址字节和一个 16 位的数据字。读写操作可以读写单个寄存器, 在 SCK 的上升沿读出数据位, 所以主机必须在下降沿刷新数据。SCI 的字节数据总是高位在前低位在后的。第一个字节指令字节, 只有 2 个指令, 也就是读和写, 读为 0X03, 写为 0X02。

一个典型的 SCI 读时序如图所示:



从上图可以看出，向 VS1053 读取数据，通过先拉低 XCS (VS_XCS)，然后发送读指令 (0X03)，再发送一个地址，最后，我们在 SO 线 (VS_MISO) 上就可以读到输出的数据了。而同时 SI (VS_MOSI) 上的数据将被忽略。

看完了 SCI 的读，我们再来看看 SCI 的写时序，如图所示：



写时序图和读时序图基本类似，都是先发指令再发地址。不过写时序中，我们的指令是写指令 (0X02)，并且数据是通过 SI 写入 VS1053 的，SO 则一直维持低电平。细心的读者可能发现了，在这两个图中，DREQ 信号上都产生了一个短暂的低脉冲，也就是执行时间。这个不难理解，我们在写入和读出 VS1053 的数据之后，它需要一些时间来处理内部的事情，这段时间是不允许外部打断的，所以，我们在 SCI 操作之前，最好判断一下 DREQ 是否为高电平，如果不是，则等待 DREQ 变为高。

了解了 VS1053 的 SPI 读写，我们再来看看 VS1053 的 SCI 寄存器，VS1053 的所有 SCI 寄存器如图所示：

SCI 寄存器				
寄存器	类型	复位值	缩写	描述
0X00	RW	0X0800	MODE	模式控制
0X01	RW	0X000C	STATUS	VS0153 状态
0X02	RW	0X0000	BASS	内置低音/高音控制
0X03	RW	0X0000	CLOCKF	时钟频率+倍频数
0X04	RW	0X0000	DECODE_TIME	解码时间长度（秒）
0X05	RW	0X0000	AUDATA	各种音频数据
0X06	RW	0X0000	WRAM	RAM 写/读
0X07	RW	0X0000	WRAMADDR	RAM 写/读的基址
0X08	R	0X0000	HDAT0	流的数据标头 0
0X09	R	0X0000	HDAT1	流的数据标头 1
0X0A	RW	0X0000	AIADDR	应用程序起始地址
0X0B	RW	0X0000	VOL	音量控制
0X0C	RW	0X0000	AICTRL0	应用控制寄存器 0
0X0D	RW	0X0000	AICTRL1	应用控制寄存器 1
0X0E	RW	0X0000	AICTRL2	应用控制寄存器 2
0X0F	RW	0X0000	AICTRL3	应用控制寄存器 3

VS1053 总共有 16 个 SCI 寄存器，这里我们不介绍全部的寄存器，仅仅介绍几个我们实验需要用到的寄存器。

首先是 MODE 寄存器，该寄存器用于控制 VS1053 的操作，是最关键的寄存器之一，该寄存器的复位值为 0x0800，其实就是默认设置为新模式。MODE 寄存器的各位描述如下所示：

位	0	1	2	3	4	5	6	7
名称	SM_DIFF	SM_LAYER12	SM_RESET	SM_CANCEL	SM_EARSPEAKER_LO	SM_TEST	SM_STREAM	SM_EARSPEAKER_HI
功能	差分	允许MPEG I&II	软件复位	取消当前文件的解码	EarSpeaker 低设定	允许SDI 测试	流模式	EarSpeaker 高设定
描述	0, 正常的同相音频 1, 左通道反相	0, 不允许 1, 允许	0, 不复位 1, 复位	0, 不取消 1, 取消	0, 关闭 1, 激活	0, 禁止 1, 允许	0, 不是 1, 是	0, 关闭 1, 激活
位	8	9	10	11	12	13	14	15
名称	SM_DACT	SM_SDIORD	SM_SDISHARE	SM_SDINEW	SM_ADPCM	-	SM_LINE1	SM_CLK_RANGE
功能	DCLK的有效边沿	SDI位顺序	共享SPI片选	VS1002本地SPI模式	ADPCM激活	-	咪/线路1选择	输入时钟范围
描述	0, 上升沿 1, 下降沿	0, MSB在前 1, MSB在后	0, 不共享 1, 共享	0, 非本地模式 1, 本地模式	0, 不激活 1, 激活	-	0, MICP 1, LINE1	0, 12..13Mhz 1, 24..26Mhz

这个寄存器，我们这里只介绍一下第 2 和第 11 位，也就是 SM_RESET 和 SM_SDINEW。其他位，我们用默认的即可。这里 SM_RESET，可以提供一次软复位，建议在每播放一首歌曲之后，软复位一次。SM_SDINEW 为模式设置位，这里我们选择的是 VS1002 新模式(本地模式)，所以设置该位为 1（默认的设置）。其他位的详细介绍，请参考 VS1053 的数据手册。

接着我们看看 BASS 寄存器，该寄存器可以用于设置 VS1053 的高低音效。

该寄存器的各位描述如图所示：

名称	位	描述
ST_AMPLITUDE	15: 12	高音控制, 1.5dB 步进 (-8..7 ,为 0 表示关闭)
ST_FREQLIMIT	11: 8	最低频限 1000Hz 步进 (0..15)
SB_AMPLITUDE	7: 4	低音加重, 1dB 步进 (0..15 ,为 0 表示关闭)
SB_FREQLIMIT	3: 0	最低频限 10Hz 步进 (2..15)

通过这个寄存器以上位的一些设置, 我们可以随意配置自己喜欢的音效 (其实就是高低音的调节)。VS1053 的 EarSpeaker 效果则由 MODE 寄存器控制, 其各位的功能在前面已列出。

接下来, 我们介绍一下 CLOCKF 寄存器, 这个寄存器用来设置时钟频率、倍频等相关信息, 该寄存器的各位描述如图所示:

CLOCKF 寄存器			
位	15:13	12:11	10:0
名称	SC_MULT	SC_ADD	SC_FREQ
描述	时钟倍频数	允许倍频	时钟频率
说明	$CLKI = XTALI \times (SC_MULT \times 0.5 + 1)$	倍频增量 $= SC_ADD \times 0.5$	当时钟频率不为 12.288M 时, 外部时钟的频率。 外部时钟为 12.288M 时, 此部分设置为 0 即可

我们重点看下该寄存器的 SC_FREQ, SC_FREQ 是以 4KHz 为步进的一个时钟寄存器, 当外部时钟不是 12.288M 的时候, 其计算公式为:

$$SC_FREQ = (XTALI - 8000000) / 4000$$

式中为 XTALI 的单位为 Hz。上图中 CLKI 是内部时钟频率, XTALI 是外部晶振的时钟频率。由于我们模块使用的是 12.288M 的晶振, 在这里设置此寄存器的值为 0x9800, 也就是设置内部时钟频率为输入时钟频率的 3 倍, 倍频增量为 1.0 倍。

接下来, 我们看看 DECODE_TIME 这个寄存器。该寄存器是一个存放解码时间的寄存器, 以秒钟为单位, 我们通过读取该寄存器的值, 就可以得到解码时间了。不过它是一个累计时间, 所以我们需要在每首歌播放之前把它清空一下, 以得到这首歌的准确解码时间。

HDATA0 和 HDATA1 是两个数据流头寄存器, 不同的音频文件, 读出来的值意义不一样, 我们可以通过这两个寄存器来获取音频文件的码率, 从而可以计算音频文件的总长度。这两个寄存器的详细介绍, 请参考 VS1053 的数据手册。

最后我们介绍一下 VOL 这个寄存器, 该寄存器用于控制 VS1053 的输出音

量，该寄存器可以分别控制左右声道的音量，每个声道的控制范围为 0~254，每个增量代表 0.5db 的衰减，所以该值越小，代表音量越大。比如设置为 0X0000 则音量最大，而设置为 0XFEFE 则音量最小。注意：如果设置 VOL 的值为 0XFFFF，将使芯片进入掉电模式！

关于 VS1053 的介绍，我们就介绍到这里，更详细的介绍请看 VS1053 的数据手册。

接下来我们说说如何通过最简单的步骤，来控制 VS1053 播放一般的音频文件（MP3/WMA/OGG/WAV/MIDI/AAC 等）音乐。

（1）复位 VS1053

这里包括了硬复位（拉低 RST）和软复位（设置 MODE 寄存器的 SM_RESET 位为 1），是为了让 VS1053 的状态回到原始状态，准备解码下一首歌曲。这里建议大家在每首歌曲播放之前都执行一次硬件复位和软件复位，以便更好的播放音乐。

（2）配置 VS1053 的相关寄存器

这里我们配置的寄存器包括 VS1053 的模式寄存器（MODE）、时钟寄存器（CLOCKF）、音调寄存器（BASS）、音量寄存器（VOL）等。

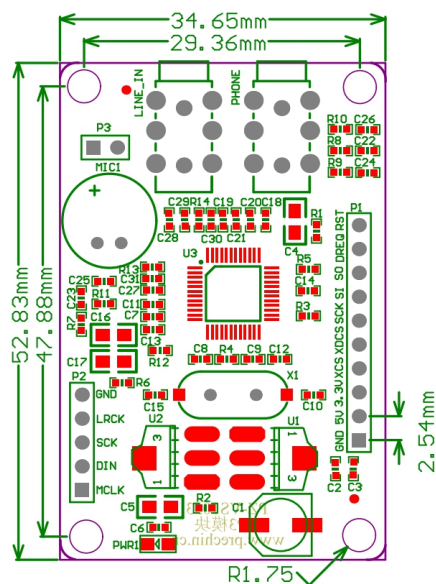
（3）发送音频数据

当经过以上两步配置以后，我们剩下来要做的事情就是往 VS1053 里面扔音频数据了，只要是 VS1053 支持的音频格式，直接往里面丢就可以了，VS1053 会自动识别，并进行播放。不过发送数据要在 DREQ 信号的控制下有序的进行，不能乱发。这个规则很简单：只要 DREQ 变高，就向 VS1053 发送 32 个字节。然后继续等待 DREQ 变高，直到音频数据发送完。

经过以上三步，我们就可以播放音乐了。这一部分就先介绍到这里。

1.1.5 结构尺寸

PZ-VS1053 MP3 模块尺寸结构如下图所示：



1.2 硬件设计

1.2.1 硬件准备

本实验所需要的硬件资源如下：

- ①F103 最小系统&玄武开发板 1 个
- ②TFTLCD 模块
- ③PZ-VS1053 MP3 模块 1 个
- ④USB 线一条（用于供电和模块与电脑串口调试助手通信）
- ⑤SD 卡一张（需在 SD 卡根目录下拷贝资料“\4--SD 卡根目录文件”内的 **MUSIC 文件夹**）
- ⑥耳机线一条

1.2.2 模块与开发板连接

PZ-VS1053 MP3 模块可通过杜邦线与 F103 最小系统&玄武开发板进行连接，该模块接口与 STM32 IO 连接关系如下所示：（PZ-VS1053 模块-->STM32 IO）

GND-->GND
5V-->5V
VS_MISO(**SO**)-->PA6

```
VS_MOSI (SI) --> PA7
VS_SCK (SCK) --> PA5
VS_XCS (XCS) --> PF7
VS_XDCS (XDCS) --> PF6
VS_DREQ (DREQ) --> PC13
VS_RST (RST) --> PE6
```

DS0 指示灯用来提示系统运行状态，KEY_UP、KEY1、KEY0 按键用来切换歌曲和音量调节，TFTLCD 用来显示当前播放音乐等信息。

注意：大家需要准备 1 个 TF 卡（在里面新建一个 MUSIC 文件夹，并存放一些歌曲在此文件夹下）和一个耳机，将 TF 卡插入到 STM32 开发板 TF 卡槽上，将耳机插入到 MP3 模块的 PHONE 接口，然后下载本实验就可以通过耳机来听歌了。

1.3 软件设计

本实验所实现的功能为：开机先检测 SD 卡是否存在，如果检测无问题，则对 VS1053 进行 RAM 测试和正弦测试，测试完后开始循环播放 SD 卡 MUSIC 文件夹里面的歌曲（**必须在 SD 卡根目录建立一个 MUSIC 文件夹，并存放歌曲在里面**），并在 TFTLCD 上显示歌曲名字、播放时间、歌曲总时间、歌曲总数目、当前歌曲的编号等信息。KEY0 用于选择下一曲，KEY1 用于选择上一曲，KEY_UP 用来调节音量。DS0 用于指示程序运行状态。

我们打开本实验工程“\1-音乐播放器实验”，可以看到我们的工程 APP 列表中多了 vs10xx.c 和 mp3player.c 源文件，以及头文件 vs10xx.h、mp3player.h 和 flac.h 等 5 个文件。本实验工程代码比较多，我们就不一一列出了，仅挑几个重要的地方进行讲解。

1.3.1 vs10xx.c

首先打开 vs10xx.c，里面的代码我们不一一贴出了，这里挑几个重要的函数给大家介绍一下，首先要介绍的是 VS_Soft_Reset，该函数用于软复位

VS1053, 其代码如下:

```
1. //软复位 VS10XX
2. void VS_Soft_Reset(void)
3. {
4.     u8 retry=0;
5.     while(VS_DQ==0); //等待软件复位结束
6.     VS_SPI_ReadWriteByte(0xff); //启动传输
7.     retry=0;
8.     while(VS_RD_Reg(SPI_MODE)!=0x0800) // 软件复位,新模式
9.     {
10.         VS_WR_Cmd(SPI_MODE,0x0804); // 软件复位,新模式
11.         delay_ms(2); //等待至少 1.35ms
12.         if(retry++>100)break;
13.     }
14.     while(VS_DQ==0); //等待软件复位结束
15.     retry=0;
16.     while(VS_RD_Reg(SPI_CLOCKF)!=0x9800) //设置 VS10XX 的时钟,3 倍频 ,1.5xADD
17.     {
18.         VS_WR_Cmd(SPI_CLOCKF,0x9800); //设置 VS10XX 的时钟,3 倍频 ,1.5xADD
19.         if(retry++>100)break;
20.     }
21.     delay_ms(20);
22. }
```

该函数比较简单,先配置一下 VS1053 的模式顺便执行软复位操作,在软复位结束之后,再设置好时钟,完成一次软复位。接下来,我们介绍一下 VS_WR_Cmd 函数,该函数用于向 VS1053 写命令,代码如下:

```
1. //向 VS10XX 写命令
2. //address:命令地址
3. //data:命令数据
4. void VS_WR_Cmd(u8 address,u16 data)
5. {
6.     while(VS_DQ==0); //等待空闲
7.     VS_SPI_SpeedLow(); //低速
8.     VS_XDCS=1;
9.     VS_XCS=0;
10.    VS_SPI_ReadWriteByte(VS_WRITE_COMMAND); //发送 VS10XX 的写命令
11.    VS_SPI_ReadWriteByte(address); //地址
12.    VS_SPI_ReadWriteByte(data>>8); //发送高八位
13.    VS_SPI_ReadWriteByte(data); //第八位
14.    VS_XCS=1;
15.    VS_SPI_SpeedHigh(); //高速
16. }
```


该函数用于向 VS1053 发送命令，这里要注意 VS1053 的写操作比读操作快（写 1/4 CLKI，读 1/7 CLKI），虽然说写寄存器最快可以到 1/4CLKI，但是经实测在 1/4CLKI 的时候会出错，所以在写寄存器的时候最好把 SPI 速度调慢点，然后在发送音频数据的时候，就可以 1/4CLKI 的速度了。有写命令的函数，当然也有读命令的函数了。VS_RD_Reg 用于读取 VS1053 的寄存器的内容。该函数代码如下：

```
1. //读 VS10XX 的寄存器
2. //address: 寄存器地址
3. //返回值: 读到的值
4. //注意不要用倍速读取,会出错
5. u16 VS_RD_Reg(u8 address)
6. {
7.     u16 temp=0;
8.     while(VS_DQ==0); //非等待空闲状态
9.     VS_SPI_SpeedLow(); //低速
10.    VS_XDCS=1;
11.    VS_XCS=0;
12.    VS_SPI_ReadWriteByte(VS_READ_COMMAND); //发送 VS10XX 的读命令
13.    VS_SPI_ReadWriteByte(address); //地址
14.    temp=VS_SPI_ReadWriteByte(0xff); //读取高字节
15.    temp=temp<<8;
16.    temp+=VS_SPI_ReadWriteByte(0xff); //读取低字节
17.    VS_XCS=1;
18.    VS_SPI_SpeedHigh(); //高速
19.    return temp;
20. }
```

该函数的作用和 VS_WR_Cmd 的作用基本相反，用于读取寄存器的值。vs10xx.c 的剩余代码、vs10xx.h 以及 flac.h 的代码，这里就不贴出来了，其中 flac.h 仅仅用来存储播放 flac 格式所需要的 patch 文件，以支持 flac 解码。大家可以打开查看他们的详细源码。

1.3.2 mp3player.c

然后我们打开 mp3player.c，该文件我们仅介绍一个函数，其他代码请看工程源码。这里要介绍的是 mp3_play_song 函数，该函数代码如下：

```
1. //播放一曲指定的歌曲
2. //返回值:0, 正常播放完成
3. //      1, 下一曲
```

```

4. //      2,上一曲
5. //      0xFF,出现错误了
6. u8 mp3_play_song(u8 *pname)
7. {
8.     FIL* fmp3;
9.     u16 br;
10.    u8 res,rval;
11.    u8 *databuf;
12.    u16 i=0;
13.    u8 key;
14.
15.    rval=0;
16.    fmp3=(FIL*)mymalloc(SRAMIN,sizeof(FIL)); //申请内存
17.    databuf=(u8*)mymalloc(SRAMIN,4096); //开辟 4096 字节的内存区域
18.    if(databuf==NULL || fmp3==NULL) rval=0xFF ; //内存申请失败.
19.    if(rval==0)
20.    {
21.        VS_Restart_Play(); //重启播放
22.        VS_Set_All(); //设置音量等信息
23.        VS_Reset_DecodeTime(); //复位解码时间
24.        res=f_typedell(pname); //得到文件后
        缀
25.        if(res==0x4c) //如果是 flac, 加载 patch
26.        {
27.            VS_Load_Patch((u16*)vs1053b_patch,VS1053B_PATCHLEN);
28.        }
29.        res=f_open(fmp3,(const TCHAR*)pname,FA_READ); //打开文件
30.        if(res==0) //打开成功.
31.        {
32.            VS_SPI_SpeedHigh(); //高速
33.            while(rval==0)
34.            {
35.                res=f_read(fmp3,databuf,4096,(UINT*)&br); //读出 4096 个字节
36.                i=0;
37.                do //主播放循环
38.                {
39.                    if(VS_Send_MusicData(databuf+i)==0) //给 VS10XX 发送音频数据
40.                    {
41.                        i+=32;
42.                    }else
43.                    {
44.                        key=KEY_Scan(0);
45.                        switch(key)
46.                        {

```

```

47.         case KEY0_PRESS:
48.             rval=1;    // 下一曲
49.             break;
50.         case KEY2_PRESS:
51.             rval=2;    // 上一曲
52.             break;
53.         case KEY_UP_PRESS: // 音量增加
54.             if(vsset.mvol<250)
55.             {
56.                 vsset.mvol+=5;
57.                 VS_Set_Vol(vsset.mvol);
58.             }else vsset.mvol=250;
59.             mp3_vol_show((vsset.mvol-100)/5);    // 音量限制在:100~250, 显示
             的时候,按照公式(vol-100)/5,显示,也就是 0~30
60.             break;
61.         case KEY1_PRESS:    // 音量减
62.             if(vsset.mvol>100)
63.             {
64.                 vsset.mvol-=5;
65.                 VS_Set_Vol(vsset.mvol);
66.             }else vsset.mvol=100;
67.             mp3_vol_show((vsset.mvol-100)/5);    // 音量限制在:100~250, 显示
             的时候,按照公式(vol-100)/5,显示,也就是 0~30
68.             break;
69.         }
70.         mp3_msg_show(fmp3->obj.objsize);// 显示信息
71.     }
72.     }while(i<4096); // 循环发送 4096 个字节
73.     if(br!=4096 || res!=0)
74.     {
75.         rval=0;
76.         break; // 读完了.
77.     }
78. }
79. f_close(fmp3);
80. }else rval=0XFF; // 出现错误
81. }
82. myfree(SRAMIN, databuf);
83. myfree(SRAMIN, fmp3);
84. return rval;
85. }

```

该函数，就是我们解码 MP3 的核心函数了，该函数在初始化 VS1053 后，根据文件格式选择是否加载 patch（如果是 flac 格式，则需要加载 patch），

最后在死循环里面等待 DREQ 信号的到来，每次 VS_DQ 变高，就通过 VS_Send_MusicData 函数向 VS1053 发送 32 个字节，直到整个文件读完。此段代码还包含了对按键的处理（音量调节、上一首、下一首）及当前播放的歌曲的一些状态（码率、播放时间、总时间）显示。

mp3player.c 的其他代码和 mp3player.h 在这里就不详细介绍了，请大家直接参考源码。

1.3.3 main.c

最后我们看看 main.c 文件的内容：

```
1.  #include "system.h"
2.  #include "SysTick.h"
3.  #include "led.h"
4.  #include "usart.h"
5.  #include "tftlcd.h"
6.  #include "key.h"
7.  #include "malloc.h"
8.  #include "sd_sdio.h"
9.  #include "flash.h"
10. #include "ff.h"
11. #include "fatfs_app.h"
12. #include "font_show.h"
13. #include "vs10xx.h"
14. #include "mp3player.h"
15.
16.
17. int main()
18. {
19.     SysTick_Init(72);
20.     NVIC_PriorityGroupConfig(NVIC_PriorityGroup_2); //中断优先级分组 分2组
21.     LED_Init();
22.     USART1_Init(115200);
23.     TFTLCD_Init(); //LCD 初始化
24.     KEY_Init();
25.     EN25QXX_Init();
26.     VS_Init(); //初始化 VS1053
27.     my_mem_init(SRAMIN); //初始化内部内存池
28.     FATFS_Init(); //为 fatfs 相关变量申请内存
29.     f_mount(fs[0], "0:", 1); //挂载 SD 卡
30.     f_mount(fs[1], "1:", 1); //挂载 SPI FLASH
```

```

31.     FRONT_COLOR=RED;
32.     while(font_init())           //检查字库
33.     {
34.         LCD_ShowString(10,10,tftlcd_data.width,tftlcd_data.height,16,"Font Error! ");
35.         delay_ms(500);
36.     }
37.     LCD_ShowFontString(10,10,tftlcd_data.width,tftlcd_data.height,"普中科技
-PRECHIN",16,0);
38.     LCD_ShowFontString(10,30,tftlcd_data.width,tftlcd_data.height,"www.prechin.net",16,0);

39.     LCD_ShowFontString(10,50,tftlcd_data.width,tftlcd_data.height,"音乐播放实验",16,0);
40.     LCD_ShowFontString(10,70,tftlcd_data.width,tftlcd_data.height,"KEY0:NEXT  KEY2:PREV",1
6,0);
41.     LCD_ShowFontString(10,90,tftlcd_data.width,tftlcd_data.height,"KEY_UP:VOL+ KEY1:VOL-",1
6,0);
42.
43.     while(1)
44.     {
45.         LED2=0;
46.         LCD_ShowFontString(10,120,tftlcd_data.width,tftlcd_data.height,"存储器测
试...",16,0);
47.         printf("Ram Test:0X%04X\r\n",VS_Ram_Test());//打印 RAM 测试结果
48.         LCD_ShowFontString(10,120,tftlcd_data.width,tftlcd_data.height,"正弦波测
试...",16,0);
49.         VS_Sine_Test();
50.         LED2=1;
51.         mp3_play();
52.     }
53. }

```

该函数先检测 SD 卡是否存在，然后执行 VS1053 的 RAM 测试和正弦测试，这两个测试结束后，调用 mp3_play 函数开始播放 SD 卡 MUSIC 文件夹里面的音乐。至此，软件部分就介绍到这里。

1.4 实验现象

首先，请先确保硬件都已经连接好：

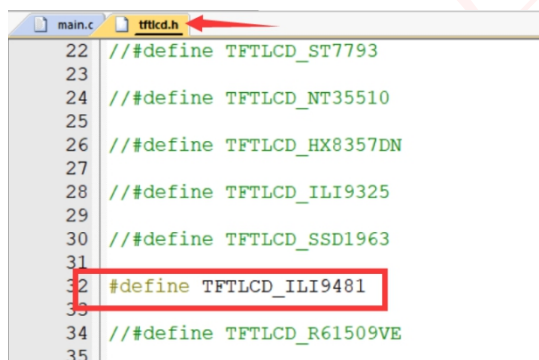
①PZ-VS1053 MP3 模块与 F103 最小系统&玄武开发板连接（可参考硬件设计小节）

②开发板插上 TFTLCD 液晶（**没有 TFTLCD 的用户可下载无彩屏时串口输出代码测试**）。

- ③使用 USB 线给开发板供电。
- ④使用一张拷贝好根目录文件的 SD 卡，音乐文件可以是 .mp3、.wav 等格式。
- ⑤使用耳机线连接好 MP3 模块。

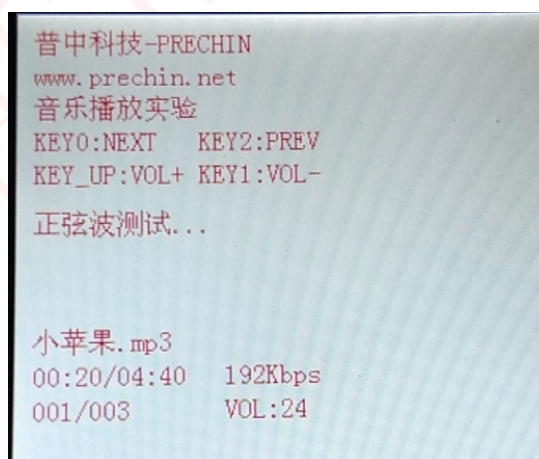
1.4.1 带 TFTLCD 测试

注意：如果使用普中 TFTLCD 触摸屏，用户需根据手上彩屏右上角或背面驱动型号来设置程序中使能的 TFTLCD 驱动。在 tftlcd.h 头文件中开始处进行设置，比如我使用的 TFTLCD 是 ILI9481，那么就要把这个宏定义放开，其他的注释掉。如下所示：



```
22 // #define TFTLCD_ST7793
23
24 // #define TFTLCD_NT35510
25
26 // #define TFTLCD_HX8357DN
27
28 // #define TFTLCD_ILI9325
29
30 // #define TFTLCD_SSD1963
31
32 #define TFTLCD_ILI9481
33
34 // #define TFTLCD_R61509VE
35
```

代码编译成功之后，我们将代码下载到 STM32 开发板上。（注意：以下的功能测试图片以玄武开发板为展示）可以看到 TFTLCD 上显示如下图所示：



```
普中科技-PRECHIN
www.prechin.net
音乐播放实验
KEY0:NEXT  KEY2:PREV
KEY_UP:VOL+ KEY1:VOL-
正弦波测试...

小苹果.mp3
00:20/04:40  192Kbps
001/003      VOL:24
```

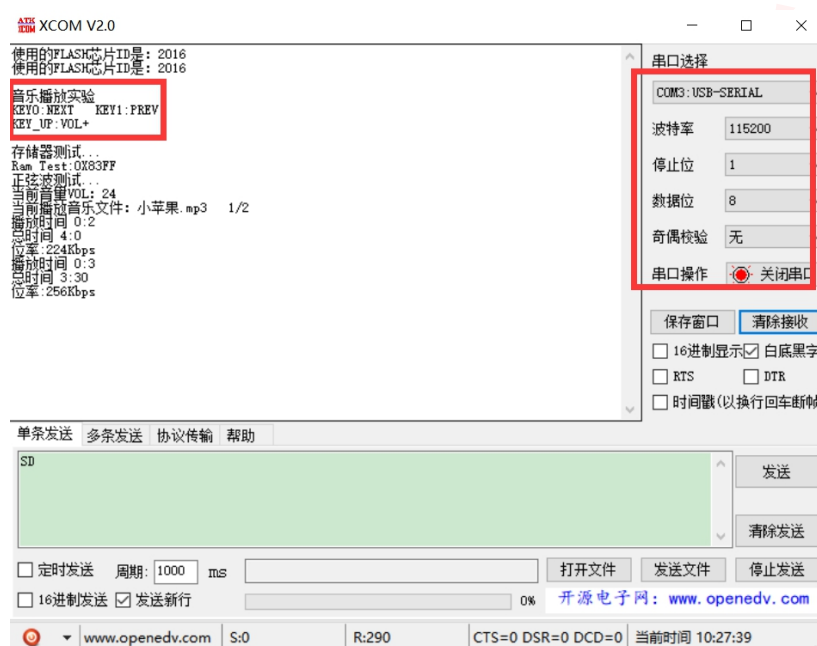
从上图可以看出，当前正在播放第 1 首歌曲，总共 3 首歌曲，歌曲名、播放时间、总时长、码率、音量等信息等也都有显示。此时 DS0 会随着音乐的播放而闪烁。

只要我们在模块的耳机端子插入耳机，就能听到歌曲的声音了。同时，我们可以通过按 KEY0 和 KEY1 来切换下一曲和上一曲，通过 KEY_UP 按键来控制音

量增加。

1.4.2 串口输出测试

使用 F103 最小系统&玄武开发板的用户可能没有 TFTLCD 彩屏，因此我们提供了串口输出的测试程序，用户可选择“无彩屏时串口输出”的实验程序下载。打开串口调试助手，设置好串口波特率等参数，现象如下：



至此，我们就完成了一个简单的 MP3 播放器了，在此基础上进一步完善，就可以做出一个比较实用的 MP3 了。大家可以自己发挥想象，做出一个你心仪的 MP3。

第 2 章 录音机实验

上一章，我们实现了一个简单的音乐播放器，本章我们将在上一章的基础上，实现一个简单的录音机，实现 WAV 录音。本章所要实现的功能为：开机先检测 SD 卡是否存在，如果检测无问题，则对 VS1053 进行 RAM 测试和正弦测试，测试完后开始实现录音及播放，在 TFTLCD 上显示录音文件名称、时间、AGC 大小等信息。KEY_UP 按键用来调节 AGC，KEY1 按键用来保存录音文件，KEY0 按键用来暂停/开始录音，触摸按键用来播放最近一次的录音文件。DS0 用于指示程序运行状态，DS1 用于指示录音状态本章分为如下几部分内容：

- 2.1 WAV 介绍
- 2.2 硬件设计
- 2.3 软件设计
- 2.4 实验现象

2.1 WAV 介绍

WAV 即 WAVE 文件，WAV 是计算机领域最常用的数字化声音文件格式之一，它是微软专门为 Windows 系统定义的波形文件格式（Waveform Audio），由于其扩展名为“*.wav”。它符合 RIFF(Resource Interchange File Format)文件规范，用于保存 Windows 平台的音频信息资源，被 Windows 平台及其应用程序所广泛支持，该格式也支持 MSADPCM，CCITT A LAW 等多种压缩运算法，支持多种音频数字，取样频率和声道，标准格式化的 WAV 文件和 CD 格式一样，也是 44.1K 的取样频率，16 位量化数字，因此在声音文件质量和 CD 相差无几。

我们 PZ-VS1053 模块板载的 VS1053 支持 2 种格式的 WAV 录音：PCM 格式或者 IMA ADPCM 格式，其中 PCM（脉冲编码调制）是最基本的 WAVE 文件格式，这种文件直接存储采样的声音数据没有经过任何的压缩。而 IAM ADPCM 则是使用了压缩算法，压缩比率为 4:1。

本章我们主要讨论 PCM，因为这个最简单。我们将利用 VS1053 实现 16 位，8Khz 采样率的单声道 WAV 录音(PCM 格式)。要想实现 WAV 录音得先了解一下 WAV 文件的格式，WAVE 文件是由若干个 Chunk 组成的。按照在文件中的出现位置包括：RIFF WAVE Chunk、Format Chunk、Fact Chunk(可选)和 Data Chunk。每个 Chunk 由块标识符、数据大小和数据三部分组成，如下图所示：

块的标识符（4BYTES）
数据大小（4BYTES）
数据

其中块标识符由 4 个 ASCII 码构成，数据大小则标出紧跟其后的数据的长度（单位为字节），注意这个长度不包含块标识符和数据大小的长度，即不包含最前面的 8 个字节。所以实际 Chunk 的大小为数据大小加 8。

首先，我们来看看 RIFF 块（RIFF WAVE Chunk），该块以“RIFF”作为标示，紧跟 wav 文件大小（该大小是 wav 文件的总大小-8），然后数据段为“WAVE”，表示是 wav 文件。RIFF 块的 Chunk 结构如下：

```
1. //RIFF 块
2. typedef __packed struct
3. {
4.     u32 ChunkID; //chunk id;这里固定为"RIFF",即 0X46464952
```

```

5.      u32 ChunkSize ; //集合大小;文件总大小-8
6.      u32 Format; //格式;WAVE,即 0X45564157
7.      }ChunkRIFF ;

```

接着,我们看看 Format 块 (Format Chunk), 该块以 “fmt” 作为标示 (注意有个空格!), 一般情况下, 该段的大小为 16 个字节, 但是有些软件生成的 wav 格式, 该部分可能有 18 个字节, 含有 2 个字节的附加信息。Format 块的 Chunk 结构如下:

```

1.      //fmt 块
2.      typedef __packed struct
3.      {
4.          u32 ChunkID; //chunk id;这里固定为"fmt ",即 0X20746D66
5.          u32 ChunkSize ; //子集合大小(不包括 ID 和 Size);这里为:20.
6.          u16 AudioFormat; //音频格式;0X10,表示线性 PCM;0X11 表示 IMA ADPCM
7.          u16 NumOfChannels; //通道数量;1,表示单声道;2,表示双声道;
8.          u32 SampleRate; //采样率;0X1F40,表示 8Khz
9.          u32 ByteRate; //字节速率;
10.         u16 BlockAlign; //块对齐(字节);
11.         u16 BitsPerSample; //单个采样数据大小;4 位 ADPCM,设置为 4
12.     }ChunkFMT;

```

接下来,我们再看看 Fact 块 (Fact Chunk), 该块为可选块, 以 “fact” 作为标示, 不是每个 WAV 文件都有, 在非 PCM 格式的文件中, 一般会在 Format 结构后面加入一个 Fact 块, 该块 Chunk 结构如下:

```

1.      //fact 块
2.      typedef __packed struct
3.      {
4.          u32 ChunkID; //chunk id;这里固定为"fact",即 0X74636166;
5.          u32 ChunkSize ; //子集合大小(不包括 ID 和 Size);这里为:4.
6.          u32 DataFactSize; //数据转换为 PCM 格式后的大小
7.      }ChunkFACT;

```

DataFactSize 是这个 Chunk 中最重要的数据, 如果这是某种压缩格式的声音文件, 那么从这里就可以知道他解压缩后的大小。对于解压时的计算会有很大的好处! 不过本章我们使用的是 PCM 格式, 所以不存在这个块。

最后,我们来看看数据块 (Data Chunk), 该块是真正保存 wav 数据的地方, 以 “data” 作为该 Chunk 的标示。然后是数据的大小。紧接着就是 wav 数据。根据 Format Chunk 中的声道数以及采样 bit 数, wav 数据的 bit 位置可以分成如下图所示的几种形式:

单声道	取样 1	取样 2	取样 3	取样 4
8 位量化	声道 0	声道 0	声道 0	声道 0
双声道	取样 1		取样 2	
8 位量化	声道 0(左)	声道 1(右)	声道 0(左)	声道 1(右)
单声道	取样 1		取样 2	
16 位量化	声道 0(低字节)	声道 0(高字节)	声道 0(低字节)	声道 0(高字节)
双声道	取样 1			
16 位量化	声道 0 (左, 低字节)	声道 0 (左, 高字节)	声道 1 (右, 低字节)	声道 1 (右, 高字节)

本章，我们采用的是 16 位，单声道，所以每个取样为 2 个字节，低字节在前，高字节在后。数据块的 Chunk 结构如下：

```

1.  //data 块
2.  typedef __packed struct
3.  {
4.      u32 ChunkID; //chunk id;这里固定为"data",即 0X61746164
5.      u32 ChunkSize ; //子集合大小(不包括 ID 和 Size);文件大小-60.
6.  }ChunkDATA;

```

通过以上学习，我们对 WAVE 文件有了个大概了解。接下来，我们看看如何使用 VS1053 实现 WAV（PCM 格式）录音。

激活 PCM 录音

VS1053 激活 PCM 录音需要设置的寄存器和相关位如下图所示：

寄存器	位域	说明
SCI_MODE	2, 12, 14	开始 ADPCM 模式，选择：咪/线路1
SCI_AICTRL0	15..0	采样率 8000..48000 Hz（在录音启动时读取的）
SCI_AICTRL1	15..0	录音增益 (1024 = 1×) 或 0 是自动增益控制 (AGC)
SCI_AICTRL2	15..0	自动增益放大器的最大值 (1024 = 1×, 65535 = 64×)
SCI_AICTRL3	1..0 2 15..3	0=联合立体声(共用 AGC), 1=双声道(各自的 AGC), 2=左通道, 3=右通道 0=IMA ADPCM 模式, 1=线性 PCM 模式 保留，设置为 0

通过设置 SCI_MODE 寄存器的 2、12、14 位，来激活 PCM 录音，SCI_MODE 的各位描述可在上一章 VS1053 介绍内查看(也可以参考 VS1053 的数据手册)。SCI_AICTRL0 寄存器用于设置采样率，我们本章用的是 8K 的采样率，所以设置这个值为 8000 即可。SCI_AICTRL1 寄存器用于设置 AGC，1024 相当于数字增加 1，这里建议大家设置 AGC 在 4（4*1024）左右比较合适。SCI_AICTRL2 用于设置自动 AGC 的时候的最大值，当设置为 0 的时候表示最大 64(65536)，这个大家按自己的需要设置即可。最后，SCI_AICTRL3，我们本章用到的是咪头线性 PCM 单声道录音，所以设置该寄存器值为 6。

通过这几个寄存器的设置，我们就激活 VS1053 的 PCM 录音了。不过，

VS1053 的 PCM 录音有一个小 BUG，必须通过加载 patch 才能解决，如果不加载 patch，那么 VS1053 是不输出 PCM 数据的，VLSI 提供了我们这个 patch，只需要通过软件加载即可。

读取 PCM 数据

在激活了 PCM 录音之后，SCI_HDAT0 和 SCI_HDAT1 有了新的功能。VS1053 的 PCM 采样缓冲区由 1024 个 16 位数据组成，如果 SCI_HDAT1 大于 0，则说明可以从 SCI_HDAT0 读取至少 SCI_HDAT1 个 16 位数据，如果数据没有被及时读取，那么将溢出，并返回空的状态。

注意，如果 $\text{SCI_HDAT1} \geq 896$ ，最好等待缓冲区溢出，以免数据混叠。所以，对我们来说，只需要判断 SCI_HDAT1 的值非零，然后从 SCI_HDAT0 读取对应长度的数据，即完成一次数据读取，以此循环，即可实现 PCM 数据的持续采集。

最后，我们看看本章实现 WAV 录音需要经过哪些步骤：

(1) 设置 VS1053 PCM 采样参数

这一步，我们要设置 PCM 的格式（线性 PCM）、采样率（8K）、位数（16 位）、通道数（单声道）等重要参数，同时还要选择采样通道（咪头），还包括 AGC 设置等。可以说这里的设置直接决定了我们 wav 文件的性质。

(2) 激活 VS1053 的 PCM 模式，加载 patch

通过激活 VS1053 的 PCM 格式，让其开始 PCM 数据采集，同时，由于 VS1053 的 BUG，我们需要加载 patch，以实现正常的 PCM 数据接收。

(3) 创建 WAV 文件，并保存 wav 头

在前两部设置成功之后，我们即可正常的从 SCI_HDAT0 读取我们需要的 PCM 数据了，不过在这之前，我们需要先在创建一个新的文件，并写入 wav 头，然后才能开始写入我们的 PCM 数据。

(4) 读取 PCM 数据

经过前面几步的处理，这一步就比较简单了，只需要不停的从 SCI_HDAT0 读取数据，然后存入 wav 文件即可，不过这里我们还需要做文件大小统计，在最后的时候写入 wav 头里面。

(5) 计算整个文件大小，重新保存 wav 头并关闭文件

在结束录音的时候，我们必须知道本次录音的大小（数据大小和整个文件大

小)，然后更新 wav 头，重新写入文件，最后因为 FATFS，在文件创建之后，必须调用 `f_close`，文件才会真正体现在文件系统里面，否则是不会写入的！所以最后还需要调用 `f_close`，以保存文件。

2.2 硬件设计

本实验使用到硬件资源如下：

- (1) DS0 和 DS1 指示灯
- (2) KEY_UP、KEY1、KEY0 和触摸按键
- (3) 串口 1
- (4) TFTLCD 模块
- (5) SD 卡
- (6) 外部 FLASH
- (7) VS1053

这些模块电路在前面章节都介绍过，这里就不多说。VS1053 与 STM32 连接在介绍 VS1053 时也讲解了，这里不再重复。

DS0 指示灯用来提示系统运行状态，DS1 指示灯用来指示录音暂停与开始，KEY_UP 按键用来调节 AGC，KEY1 按键用来保存录音文件，KEY0 按键用来暂停/开始录音，触摸按键用来播放最近一次的录音文件。TFTLCD 用来显示当前录音等信息。

注意：大家需要准备 1 个 TF 卡和一个耳机，将 TF 卡插入到 STM32 开发板 TF 卡槽上，将耳机插入到 MP3 模块的 PHONE 接口，然后下载本实验就可以实现录音及播放。

2.3 软件设计

本实验所实现的功能为：开机先检测 SD 卡是否存在，如果检测无问题，则对 VS1053 进行 RAM 测试和正弦测试，测试完后开始实现录音及播放，在 TFTLCD 上显示录音文件名称、时间、AGC 大小等信息。KEY_UP 按键用来调节 AGC，KEY1 按键用来保存录音文件，KEY0 按键用来暂停/开始录音，触摸按键用来播放最近一次的录音文件。DS0 用于指示程序运行状态，DS1 用于指示录音状态。

我们打开本实验工程“\2-录音机实验”，可以看到我们的工程 APP 列表中多了 recorder.c 源文件以及头文件。本实验工程代码比较多，我们就不一一列出了，仅挑几个重要的地方进行讲解。

2.3.1 recorder.c

首先是设置 VS1053 进入 PCM 模式的函数：recoder_enter_rec_mode，该函数代码如下：

```
1. //激活 PCM 录音模式
2. //agc:0,自动增益.1024 相当于 1 倍,512 相当于 0.5 倍,最大值 65535=64 倍
3. void recoder_enter_rec_mode(u16 agc)
4. {
5.     //如果是 IMA ADPCM, 采样率计算公式如下:
6.     //采样率=CLKI/256*d;
7.     //假设 d=0, 并 2 倍频, 外部晶振为 12.288M. 那么 Fc=(2*12288000)/256*6=16Khz
8.     //如果是线性 PCM, 采样率直接就写采样值
9.     VS_WR_Cmd(SPI_BASS,0x0000);
10.    VS_WR_Cmd(SPI_AICTRL0,8000); //设置采样率, 设置为 8Khz
11.    VS_WR_Cmd(SPI_AICTRL1,agc); //设置增益,0,自动增益.1024 相当于 1 倍,512 相当于 0.5 倍,最大值 65535=64 倍
12.    VS_WR_Cmd(SPI_AICTRL2,0); //设置增益最大值,0,代表最大值 65536=64X
13.    VS_WR_Cmd(SPI_AICTRL3,6); //左通道(MIC 单声道输入)
14.    VS_WR_Cmd(SPI_CLOCKF,0X2000); //设置 VS10XX 的时钟,MULT:2 倍频;ADD:不允许;CLK:12.288Mhz
15.    VS_WR_Cmd(SPI_MODE,0x5804); //MIC, 录音激活
16.    delay_ms(5); //等待至少 1.35ms
17.    VS_Load_Patch((u16*)wav_plugin,40); //VS1053 的 WAV 录音需要 patch
18. }
```

该函数就是用我们前面介绍的方法，激活 VS1053 的 PCM 模式，本章，我们使用的是 8Khz 采样率，16 位单声道线性 PCM 模式，AGC 通过函数参数设置。

最后加载 patch（用于修复 VS1053 录音 BUG）。

第二个函数是初始化 wav 头的函数：recoder_wav_init，该函数代码如下：

```
1. //初始化 WAV 头.
2. void recoder_wav_init(__WaveHeader* wavhead) //初始化 WAV 头
3. {
4.     wavhead->riff.ChunkID=0X46464952; // "RIFF"
5.     wavhead->riff.ChunkSize=0; // 还未确定,最后需要计算
6.     wavhead->riff.Format=0X45564157; // "WAVE"
7.     wavhead->fmt.ChunkID=0X20746D66; // "fmt "
8.     wavhead->fmt.ChunkSize=16; // 大小为 16 个字节
9.     wavhead->fmt.AudioFormat=0X01; // 0X01,表示 PCM;0X01,表示 IMA ADPCM
10.    wavhead->fmt.NumOfChannels=1; // 单声道
11.    wavhead->fmt.SampleRate=8000; // 8Khz 采样率 采样速率
12.    wavhead->fmt.ByteRate=wavhead->fmt.SampleRate*2; // 16 位,即 2 个字节
13.    wavhead->fmt.BlockAlign=2; // 块大小,2 个字节为一个块
14.    wavhead->fmt.BitsPerSample=16; // 16 位 PCM
15.    wavhead->data.ChunkID=0X61746164; // "data"
16.    wavhead->data.ChunkSize=0; // 数据大小,还需要计算
17. }
```

该函数初始化 wav 头的绝大部分数据，这里我们设置了该 wav 文件为 8Khz 采样率，16 位线性 PCM 格式，另外由于录音还未真正开始，所以文件大小和数据大小都还是未知的，要等录音结束才能知道。该函数 __WaveHeader 结构体就是由前面介绍的三个 Chunk 组成，结构为：

```
1. //wav 头
2. typedef __packed struct
3. {
4.     ChunkRIFF riff; //riff 块
5.     ChunkFMT fmt; //fmt 块
6.     //ChunkFACT fact; //fact 块 线性 PCM,没有这个结构体
7.     ChunkDATA data; //data 块
8. } __WaveHeader;
```

最后，我们介绍 recoder_play 函数，是录音机实现的主循环函数，该函数代码如下：

```
1. //录音机
2. //所有录音文件,均保存在 SD 卡 RECORDER 文件夹内.
3. u8 recoder_play(void)
4. {
5.     u8 res;
6.     u8 key;
7.     u8 rval=0;
```

```

8.     __WaveHeader *wavhead=0;
9.     u32 sectorsize=0;
10.    FIL* f_rec=0;           //文件
11.    DIR recdir;             //目录
12.    u8 *recbuf;             //数据内存
13.    u16 w;
14.    u16 idx=0;
15.    u8 rec_sta=0;           //录音状态
16.                                //[7]:0, 没有录音;1, 有录音;
17.                                //[6:1]:保留
18.                                //[0]:0, 正在录音;1, 暂停录音;
19.    u8 *pname=0;
20.    u8 timecnt=0;           //计时器
21.    u32 recsec=0;           //录音时间
22.    u8 recagc=4;            //默认增益为 4
23.    while(f_opendir(&recdir,"0:/RECORDER"))//打开录音文件夹
24.    {
25.        LCD_ShowFontString(10,230,240,16,"RECORDER 文件夹错误!",16,0);
26.        delay_ms(200);
27.        LCD_Fill(10,230,240,246,WHITE);    //清除显示
28.        delay_ms(200);
29.        f_mkdir("0:/RECORDER");             //创建该目录
30.    }
31.    f_rec=(FIL *)mymalloc(SRAMIN,sizeof(FIL)); //开辟 FIL 字节的内存区域
32.    if(f_rec==NULL)rval=1; //申请失败
33.    wavhead=(__WaveHeader*)mymalloc(SRAMIN,sizeof(__WaveHeader));//开辟 __WaveHeader 字节的内存
    区域
34.    if(wavhead==NULL)rval=1;
35.    recbuf=mymalloc(SRAMIN,512);
36.    if(recbuf==NULL)rval=1;
37.    pname=mymalloc(SRAMIN,30);                //申请 30 个字节内存,类似
    "0:/RECORDER/REC00001.wav"
38.    if(pname==NULL)rval=1;
39.    if(rval==0)                                //内存申请 OK
40.    {
41.        recoder_enter_rec_mode(1024*recagc);
42.        while(VS_RD_Reg(SPI_HDAT1)>>8);        //等到 buf 较为空闲再开始
43.        recoder_show_time(recsec);              //显示时间
44.        recoder_show_agc(recagc);              //显示 agc
45.        pname[0]=0;                            //pname 没有任何文件名
46.        while(rval==0)
47.        {
48.            key=KEY_Scan(0);
49.            switch(key)

```

```

50.         {
51.             case KEY2_PRESS:    //STOP&SAVE
52.                 if(rec_sta&0X80)//有录音
53.                 {
54.                     wavhead->riff.ChunkSize=sectorsize*512+36;    //整个文件的大小-8;
55.                     wavhead->data.ChunkSize=sectorsize*512;        //数据大小
56.                     f_lseek(f_rec,0);                            //偏移到文件头.
57.                     f_write(f_rec,(const void*)wavhead,sizeof(__WaveHeader),&bw);//写入
                        头数据
58.                     f_close(f_rec);
59.                     sectorsize=0;
60.                 }
61.                 rec_sta=0;
62.                 recsec=0;
63.                 LED2=1;                                          //关闭 DS1
64.                 LCD_Fill(10,230,240,246,WHITE); //清除显示,清除之前显示的录音文件
                        名
65.                 recoder_show_time(recsec);    //显示时间
66.                 break;
67.             case KEY0_PRESS:    //REC/PAUSE
68.                 if(rec_sta&0X01)//原来是暂停,继续录音
69.                 {
70.                     rec_sta&=0XFE;//取消暂停
71.                 }else if(rec_sta&0X80)//已经在录音了,暂停
72.                 {
73.                     rec_sta|=0X01; //暂停
74.                 }else          //还没开始录音
75.                 {
76.                     rec_sta|=0X80; //开始录音
77.                     recoder_new_pathname(pname);        //得到新的名字
78.                     LCD_ShowFontString(10,230,240,16,pname+11,16,0);    //显示当前录音文
                        件名字
79.                     recoder_wav_init(wavhead);          //初始化 wav 数据
80.                     res=f_open(f_rec,(const TCHAR*)pname, FA_CREATE_ALWAYS | FA_WRITE);

81.                     if(res)          //文件创建失败
82.                     {
83.                         rec_sta=0;    //创建文件失败,不能录音
84.                         rval=0XFE;    //提示是否存在 SD 卡
85.                     }else res=f_write(f_rec,(const void*)wavhead,sizeof(__WaveHeader),&
                        bw);//写入头数据
86.                     }
87.                     if(rec_sta&0X01)LED2=0; //提示正在暂停
88.                     else LED2=1;

```

```

89.             break;
90.             case KEY_UP_PRESS: //AGC+
91.             case KEY1_PRESS: //AGC-
92.                 if(key==KEY_UP_PRESS)recagc++;
93.                 else if(recagc)recagc--;
94.                 if(recagc>15)recagc=15; //范围限定为 0~15.0,自动 AGC.其他 AGC
           倍数
95.                 recoder_show_agc(recagc);
96.                 VS_WR_Cmd(SPI_AICTRL1,1024*recagc); //设置增益,0,自动增益.1024 相当于 1
           倍,512 相当于 0.5 倍
97.             break;
98.         }
99.         ////////////////////////////////////////////
100.    //读取数据
101.        if(rec_sta==0X80)//已经在录音了
102.        {
103.            w=VS_RD_Reg(SPI_HDAT1);
104.            if((w>=256)&&(w<896))
105.            {
106.                idx=0;
107.                while(idx<512) //一次读取 512 字节
108.                {
109.                    w=VS_RD_Reg(SPI_HDAT0);
110.                    recbuf[idx++]=w&0XFF;
111.                    recbuf[idx++]=w>>8;
112.                }
113.                res=f_write(f_rec,recbuf,512,&bw);//写入文件
114.                if(res)
115.                {
116.                    printf("err:%d\r\n",res);
117.                    printf("bw:%d\r\n",bw);
118.                    break;//写入出错.
119.                }
120.                sectorsize++;//扇区数增加 1,约为 32ms
121.            }
122.        }else//没有开始录音,则检测触摸按键
123.        {
124.            if(Touch_Key_Scan(0)&&pname[0])//如果触摸按键被按下,且 pname 不为空
125.            {
126.                LCD_ShowFontString(10,230,240,16,"播放:",16,0);
127.                LCD_ShowFontString(10+40,230,240,16,pname+11,16,0); //显示当播放的文件名
           字
128.                rec_play_wav(pname); //播放 pname

```

```

129.          LCD_Fill(10,230,240,246,WHITE);          //清除显示,清除之前显示的录音
           文件名
130.          recoder_enter_rec_mode(1024*recagc);      //重新进入录音模式
131.          while(VS_RD_Reg(SPI_HDAT1)>>8);          //等到 buf 较为空闲再开
           始
132.          recoder_show_time(recsec);                //显示时间
133.          recoder_show_agc(recagc);                 //显示 agc
134.      }
135.          delay_ms(5);
136.          timecnt++;
137.          if((timecnt%20)==0)LED1=!LED1;//LED 闪烁
138.      }
139.  //////////////////////////////////////
140.          if(recsec!=(sectorsize*4/125))//录音时间显示
141.      {
142.          LED1=!LED1;//LED 闪烁
143.          recsec=sectorsize*4/125;
144.          recoder_show_time(recsec);//显示时间
145.      }
146.  }
147.  }
148.  myfree(SRAMIN,wavhead);
149.  myfree(SRAMIN,recbuf);
150.  myfree(SRAMIN,f_rec);
151.  myfree(SRAMIN,pname);
152.  return rval;
153. }

```

该函数实现了我们在硬件设计时介绍的功能，我们就不详细介绍了。
recorder.c 的其他代码和 recorder.h 的代码我们这里就不再贴出了，请大家参考本实验的源码。

2.3.2 main.c

最后我们看看主函数：

```

1.  #include "system.h"
2.  #include "SysTick.h"
3.  #include "led.h"
4.  #include "usart.h"
5.  #include "tftlcd.h"
6.  #include "key.h"
7.  #include "malloc.h"
8.  #include "sd_sdio.h"

```

```

9.    #include "flash.h"
10.   #include "ff.h"
11.   #include "fatfs_app.h"
12.   #include "font_show.h"
13.   #include "vs10xx.h"
14.   #include "touch_key.h"
15.   #include "recorder.h"
16.
17.
18.   int main()
19.   {
20.       SysTick_Init(72);
21.       NVIC_PriorityGroupConfig(NVIC_PriorityGroup_2); //中断优先级分组 分2组
22.       LED_Init();
23.       USART1_Init(115200);
24.       TFTLCD_Init(); //LCD 初始化
25.       KEY_Init();
26.       Touch_Key_Init(6);
27.       EN25QXX_Init();
28.       VS_Init(); //初始化 VS1053
29.       my_mem_init(SRAMIN); //初始化内部内存池
30.       FATFS_Init(); //为 fatfs 相关变量申请内存
31.       f_mount(fs[0], "0:", 1); //挂载 SD 卡
32.       f_mount(fs[1], "1:", 1); //挂载 SPI FLASH
33.       FRONT_COLOR=RED;
34.       while(font_init()) //检查字库
35.       {
36.           LCD_ShowString(10,10,tftlcd_data.width,tftlcd_data.height,16,"Font Error! ");
37.           delay_ms(500);
38.       }
39.       LCD_ShowFontString(10,10,tftlcd_data.width,tftlcd_data.height,"普中科技
-PRECHIN",16,0);
40.       LCD_ShowFontString(10,30,tftlcd_data.width,tftlcd_data.height,"www.prechin.net",16,0);
41.       LCD_ShowFontString(10,50,tftlcd_data.width,tftlcd_data.height,"WAV 录音机实验",16,0);
42.       LCD_ShowFontString(10,70,tftlcd_data.width,tftlcd_data.height,"KEY0:REC/PAUSE",16,0);
43.       LCD_ShowFontString(10,90,tftlcd_data.width,tftlcd_data.height,"KEY2:STOP&SAVE",16,0);
44.       LCD_ShowFontString(10,110,tftlcd_data.width,tftlcd_data.height,"KEY_UP:AGC+ KEY1:AGC-",
16,0);
45.       LCD_ShowFontString(10,130,tftlcd_data.width,tftlcd_data.height,"触摸按
键:Play The File",16,0);
46.
47.       while(1)
48.       {

```



```

49.         LED2=0;
50.         LCD_ShowFontString(10,160,tftlcd_data.width,tftlcd_data.height,"存储器测
           试...",16,0);
51.         printf("Ram Test:0X%04X\r\n",VS_Ram_Test());//打印 RAM 测试结果
52.         LCD_ShowFontString(10,160,tftlcd_data.width,tftlcd_data.height,"正弦波测
           试...",16,0);
53.         VS_Sine_Test();
54.         LCD_Fill(10,160,240,176,WHITE);
55.         LED2=1;
56.         recoder_play();
57.     }
58. }

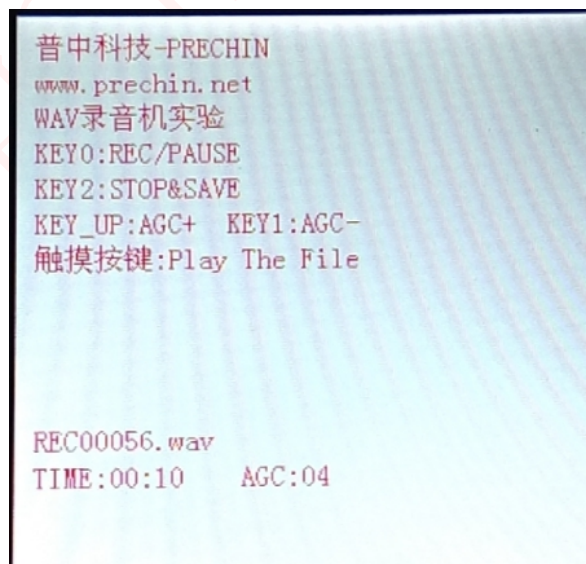
```

该函数先检测字库是否存在，然后执行 VS1053 的 RAM 测试和正弦测试，这两个测试结束后，调用 recoder_play 函数开始执行录音及播放功能。至此，软件部分就介绍到这里。

2.4 实验现象

2.4.1 带 TFTLCD 显示

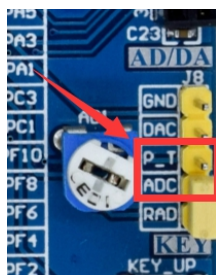
将工程程序编译下载到开发板内，**注意一定要插上 SD 卡**。可以看到 TFTLCD 上显示如下界面：



此时，我们按下 KEY0 就开始录音了，此时看到屏幕显示录音文件的名称以及录音时长。在录音的时候按下 KEY0 则执行暂停/继续录音的切换，通过 DS1 指示录音暂停，按 KEY_UP 可以调节 AGC，AGC 越大越灵敏，不过不建议设置太

大，因为这可能导致失真。通过按下 KEY1，可以停止当前录音，并保存录音文件。在完成一次录音文件保存之后，我们可以通过按触摸按键，来实现播放这个录音文件（即播放最近一次的录音文件），实现试听。

对于玄武开发板使用触摸按键时，要将 J8 端子上的黄色跳线帽短接到 P_T 和 ADC 引脚。如下所示：



我们将开发板的录音文件放到电脑上面，可以通过属性查看录音文件的属性，如下图所示：



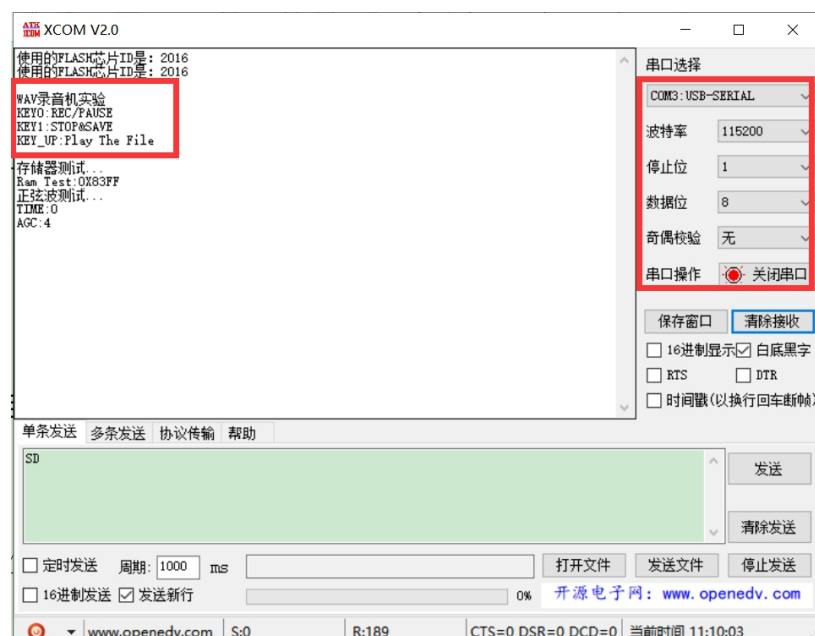
这和我们预期的效果一样，通过电脑端的播放器可以直接播放我们所录的音频。经实测效果还是不错的。

2.4.2 串口输出显示

使用 F103 最小系统&玄武开发板的用户可能没有 TFTLCD 彩屏，因此我们提供了串口输出的测试程序，用户可选择“无彩屏时串口输出”的实验程序下载。

打开串口调试助手，设置好串口波特率等参数，现象如下：

注意：对于 F103 最小系统开发板没有触摸按键，我们已经取消 KEY_UP 键调节 AGC 功能，使用 KEY_UP 键来代替触摸按键，进行播放录音文件。



3 其他

(1) 购买地址（普中授权店铺）

<http://www.prechin.net/forum.php?mod=viewthread&tid=38746&extra=>

(2) 资料下载

<http://prechin.net/forum.php?mod=viewthread&tid=35264&extra=page%3D1>

(3) 技术支持

普中官网：www.prechin.cn

普中论坛：www.prechin.net

技术电话：0755-21509063（转技术）